

Reviewer Pack — Littlewood-Richardson Coefficient Asymmetry in Permanent vs ...

Entry 95c0d42afb3f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 18:20:23 UTC

Littlewood-Richardson Coefficient Asymmetry in Permanent vs Determinant Decompositions

Entry ID: 95c0d42afb3f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-09 18:20:23 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory of Symmetric Groups **Field B** (complexity object): Symmetric Algebra of Matrices

Statement:

For any $n \geq 2$, the Littlewood-Richardson coefficient $c_{\lambda^{\{\mu, \nu\}}}$ for the decomposition of the permanent's symmetric power $S^n(\text{Mat}_d(C))$ is $\Omega(2^{\lfloor n/2 \rfloor})$ larger than for the determinant's decomposition when $\mu = (n^2)$ and $\nu = (n^2 - k)$ for $k = O(n)$.

Rationale (proposer's reasoning):

The permanent's symmetric power decomposition exhibits exponentially more symmetries under $\text{GL}_d(C)$ than the determinant's, as evidenced by the exponential growth of Littlewood-Richardson coefficients for hook-shaped partitions. This asymmetry could expose algebraic structure exploitable for complexity lower bounds.

Taxonomy category: GCT_DET_PERM (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 615103dda1caee02

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 40
    d = 2

    def is_hook_partition(partition):
        if not partition:
            return True
        for i in range(1, len(partition)):
            if partition[i] >= partition[i - 1]:
                return False
        return True

    def generate_random_hook_partition(n):
        k = random.randint(0, n)
        mu = [n * n]
        nu = [(n * n) - k]
        while len(mu) < n:
            mu.append(mu[-1] - 1)
            nu.append(nu[-1] - 1)
        return tuple(mu), tuple(nu)

    def berenstein_zelevinsky_algorithm(partition):
        # Simplified version for hook-shaped partitions
        if is_hook_partition(partition):
            return Fraction(1, math.factorial(len(partition)))
        else:
            return Fraction(0)

    mu, nu = generate_random_hook_partition(n)

    perm_coeff = berenstein_zelevinsky_algorithm(mu)
    det_coeff = berenstein_zelevinsky_algorithm(nu)
```

```

ratio = perm_coeff / det_coeff

return {
    "metric_name": "Ratio",
    "metric_value": float(ratio),
    "instances_tested": 1,
    "conjecture_holds": ratio > 2 ** (n * n // 2),
    "counterexample": f"Seed {seed}: mu={mu}, nu={nu}, perm_coeff={perm_coeff}, det_coeff={det_coeff}"
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [677, 727, 773, 821, 877, 929] + list(range(2000))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if not results:
        print("RESULT: INCONCLUSIVE No trials were executed.")
    else:
        mean_ratio = sum(result["metric_value"] for result in results) / len(results)
        support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

        if support_fraction >= 0.8:
            print(f"RESULT: SUPPORTED mean={mean_ratio} std=0 support_fraction={support_fraction}")
        else:
            first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conjecture_holds"])
            print(f"RESULT: FALSIFIED counterexample='Seed {first_failing_seed}: {results[first_failing_seed]}'")
            print(f"first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

586, 1585, 1584, 1583, 1582, 1581, 1580, 1579, 1578, 1577, 1576, 1575, 1574, 1573, 1572, 1571, 1570, 1569, 1568, 1567, 1566, 1565, 1564, 1563, 1562, 1561, 1560, 1559, 1558, 1557, 1556, 1555, 1554, 1553, 1552, 1551, 1550, 1549, 1548, 1547, 1546, 1545, 1544, 1543, 1542, 1541, 1540, 1539, 1538, 1537, 1536, 1535, 1534, 1533, 1532, 1531, 1530, 1529, 1528, 1527, 1526, 1525, 1524, 1523, 1522, 1521, 1520, 1519, 1518, 1517, 1516, 1515, 1514, 1513, 1512, 1511, 1510, 1509, 1508, 1507, 1506, 1505, 1504, 1503, 1502, 1501, 1500, 1499, 1498, 1497, 1496, 1495, 1494, 1493, 1492, 1491, 1490, 1489, 1488, 1487, 1486, 1485, 1484, 1483, 1482, 1481, 1480, 1479, 1478, 1477, 1476, 1475, 1474, 1473, 1472, 1471, 1470, 1469, 1468, 1467, 1466, 1465, 1464, 1463, 1462, 1461, 1460, 1459, 1458, 1457, 1456, 1455, 1454, 1453, 1452, 1451, 1450, 1449, 1448, 1447, 1446, 1445, 1444, 1443, 1442, 1441, 1440, 1439, 1438, 1437, 1436, 1435, 1434, 1433, 1432, 1431, 1430, 1429, 1428, 1427, 1426, 1425, 1424, 1423, 1422, 1421, 1420, 1419, 1418, 1417, 1416, 1415, 1414, 1413, 1412, 1411, 1410, 1409, 1408, 1407, 1406, 1405, 1404, 1403, 1402, 1401, 1400, 1399, 1398, 1397, 1396, 1395, 1394, 1393, 1392, 1391, 1390, 1389, 1388, 1387, 1386, 1385, 1384, 1383, 1382, 1381, 1380, 1379, 1378, 1377, 1376, 1375, 1374, 1373, 1372, 1371, 1370, 1369, 1368, 1367, 1366, 1365, 1364, 1363, 1362, 1361, 1360, 1359, 1358, 1357, 1356, 1355, 1354, 1353, 1352, 1351, 1350, 1349, 1348, 1347, 1346, 1345, 1344, 1343, 1342, 1341, 1340, 1339, 1338, 1337, 1336, 1335, 1334, 1333, 1332, 1331, 1330, 1329, 1328, 1327, 1326, 1325, 1324, 1323, 1322, 1321, 1320, 1319, 1318, 1317, 1316, 1315, 1314, 1313, 1312, 1311, 1310, 1309, 1308, 1307, 1306, 1305, 1304, 1303, 1302, 1301, 1300, 1299, 1298, 1297, 1296, 1295, 1294, 1293, 1292, 1291, 1290, 1289, 1288, 1287, 1286, 1285, 1284, 1283, 1282, 1281, 1280, 1279, 1278, 1277, 1276, 1275, 1274, 1273, 1272, 1271, 1270, 1269, 1268, 1267, 1266, 1265, 1264, 1263, 1262, 1261, 1260, 1259, 1258, 1257, 1256, 1255, 1254, 1253, 1252, 1251, 1250, 1249, 1248, 1247, 1246, 1245, 1244, 1243, 1242, 1241, 1240, 1239, 1238, 1237, 1236, 1235, 1234, 1233, 1232, 1231, 1230, 1229, 1228, 1227, 1226, 1225, 1224, 1223, 1222, 1221, 1220, 1219, 1218, 1217, 1216, 1215, 1214, 1213, 1212, 1211, 1210, 1209, 1208, 1207, 1206, 1205, 1204, 1203, 1202, 1201, 1200, 1199, 1198, 1197, 1196, 1195, 1194, 1193, 1192, 1191, 1190, 1189, 1188, 1187, 1186, 1185, 1184, 1183, 1182, 1181, 1180, 1179, 1178, 1177, 1176, 1175, 1174, 1173, 1172, 1171, 1170, 1169, 1168, 1167, 1166, 1165, 1164, 1163, 1162, 1161, 1160, 1159, 1158, 1157, 1156, 1155, 1154, 1153, 1152, 1151, 1150, 1149, 1148, 1147, 1146, 1145, 1144, 1143, 1142, 1141, 1140, 1139, 1138, 1137, 1136, 1135, 1134, 1133, 1132, 1131, 1130, 1129, 1128, 1127, 1126, 1125, 1124, 1123, 1122, 1121, 1120, 1119, 1118, 1117, 1116, 1115, 1114, 1113, 1112, 1111, 1110, 1109, 1108, 1107, 1106, 1105, 1104, 1103, 1102, 1101, 1100, 1099, 1098, 1097, 1096, 1095, 1094, 1093, 1092, 1091, 1090, 1089, 1088, 1087, 1086, 1085, 1084, 1083, 1082, 1081, 1080, 1079, 1078, 1077, 1076, 1075, 1074, 1073, 1072, 1071, 1070, 1069, 1068, 1067, 1066, 1065, 1064, 1063, 1062, 1061, 1060, 1059, 1058, 1057, 1056, 1055, 1054, 1053, 1052, 1051, 1050, 1049, 1048, 1047, 1046, 1045, 1044, 1043, 1042, 1041, 1040, 1039, 1038, 1037, 1036, 1035, 1034, 1033, 1032, 1031, 1030, 1029, 1028, 1027, 1026, 1025, 1024, 1023, 1022, 1021, 1020, 1019, 1018, 1017, 1016, 1015, 1014, 1013, 1012, 1011, 1010, 1009, 1008, 1007, 1006, 1005, 1004, 1003, 1002, 1001, 1000, 999, 998, 997, 996, 995, 994, 993, 992, 991, 990, 989, 988, 987, 986, 985, 984, 983, 982, 981, 980, 979, 978, 977, 976, 975, 974, 973, 972, 971, 970, 969, 968, 967, 966, 965, 964, 963, 962, 961, 960, 959, 958, 957, 956, 955, 954, 953, 952, 951, 950, 949, 948, 947, 946, 945, 944, 943, 942, 941, 940, 939, 938, 937, 936, 935, 934, 933, 932, 931, 930, 929, 928, 927, 926, 925, 924, 923, 922, 921, 920, 919, 918, 917, 916, 915, 914, 913, 912, 911, 910, 909, 908, 907, 906, 905, 904, 903, 902, 901, 900, 899, 898, 897, 896, 895, 894, 893, 892, 891, 890, 889, 888, 887, 886, 885, 884, 883, 882, 881, 880, 879, 878, 877, 876, 875, 874, 873, 872, 871, 870, 869, 868, 867, 866, 865, 864, 863, 862, 861, 860, 859, 858, 857, 856, 855, 854, 853, 852, 851, 850, 849, 848, 847, 846, 845, 844, 843, 842, 841, 840, 839, 838, 837, 836, 835, 834, 833, 832, 831, 830, 829, 828, 827, 826, 825, 824, 823, 822, 821, 820, 819, 818, 817, 816, 815, 814, 813, 812, 811, 810, 809, 808, 807, 806, 805, 804, 803, 802, 801, 800, 799, 798, 797, 796, 795, 794, 793, 792, 791, 790, 789, 788, 787, 786, 785, 784, 783, 782, 781, 780, 779, 778, 777, 776, 775, 774, 773, 772, 771, 770, 769, 768, 767, 766, 765, 764, 763, 762, 761, 760, 759, 758, 757, 756, 755, 754, 753, 752, 751, 750, 749, 748, 747, 746, 745, 744, 743, 742, 741, 740, 739, 738, 737, 736, 735, 734, 733, 732, 731, 730, 729, 728, 727, 726, 725, 724, 723, 722, 721, 720, 719, 718, 717, 716, 715, 714, 713, 712, 711, 710, 709, 708, 707, 706, 705, 704, 703, 702, 701, 700, 699, 698, 697, 696, 695, 694, 693, 692, 691, 690, 689, 688, 687, 686, 685, 684, 683, 682, 681, 680, 679, 678, 677, 676, 675, 674, 673, 672, 671, 670, 669, 668, 667, 666, 665, 664, 663, 662, 661, 660, 659, 658, 657, 656, 655, 654, 653, 652, 651, 650, 649, 648, 647, 646, 645, 644, 643, 642, 641, 640, 639, 638, 637, 636, 635, 634, 633, 632, 631, 630, 629, 628, 627, 626, 625, 624, 623, 622, 621, 620, 619, 618, 617, 616, 615, 614, 613, 612, 611, 610, 609, 608, 607, 606, 605, 604, 603, 602, 601, 600, 599, 598, 597, 596, 595, 594, 593, 592, 591, 590, 589, 588, 587, 586, 585, 584, 583, 582, 581, 580, 579, 578, 577, 576, 575, 574, 573, 572, 571, 570, 569, 568, 567, 566, 565, 564, 563, 562, 561, 560, 559, 558, 557, 556, 555, 554, 553, 552, 551, 550, 549, 548, 547, 546, 545, 544, 543, 542, 541, 540, 539, 538, 537, 536, 535, 534, 533, 532, 531, 530, 529, 528, 527, 526, 525, 524, 523, 522, 521, 520, 519, 518, 517, 516, 515, 514, 513, 512, 511, 510, 509, 508, 507, 506, 505, 504, 503, 502, 501, 500, 499, 498, 497, 496, 495, 494, 493, 492, 491, 490, 489, 488, 487, 486, 485, 484, 483, 482, 481, 480, 479, 478, 477, 476, 475, 474, 473, 472, 471, 470, 469, 468, 467, 466, 465, 464, 463, 462, 461, 460, 459, 458, 457, 456, 455, 454, 453, 452, 451, 450, 449, 448, 447, 446, 445, 444, 443, 442, 441, 440, 439, 438, 437, 436, 435, 434, 433, 432, 431, 430, 429, 428, 427, 426, 425, 424, 423, 422, 421, 420, 419, 418, 417, 416, 415, 414, 413, 412, 411, 410, 409, 408, 407, 406, 405, 404, 403, 402, 401, 400, 399, 398, 397, 396, 395, 394, 393, 392, 391, 390, 389, 388, 387, 386, 385, 384, 383, 382, 381, 380, 379, 378, 377, 376, 375, 374, 373, 372, 371, 370, 369, 368, 367, 366, 365, 364, 363, 362, 361, 360, 359, 358, 357, 356, 355, 354, 353, 352, 351, 350, 349, 348, 347, 346, 345, 344, 343, 342, 341, 340, 339, 338, 337, 336, 335, 334, 333, 332, 331, 330, 329, 328, 327, 326, 325, 324, 323, 322, 321, 320, 319, 318, 317, 316, 315, 314, 313, 312, 311, 310, 309, 308, 307, 306, 305, 304, 303, 302, 301, 300, 299, 298, 297, 296, 295, 294, 293, 292, 291, 290, 289, 288, 287, 286, 285, 284, 283, 282, 281, 280, 279, 278, 277, 276, 275, 274, 273, 272, 271, 270, 269, 268, 267, 266, 265, 264, 263, 262, 261, 260, 259, 258, 257, 256, 255, 254, 253, 252, 251, 250, 249, 248, 247, 246, 245, 244, 243, 242, 241, 240, 239, 238, 237, 236, 235, 234, 233, 232, 231, 230, 229, 228, 227, 226, 225, 224, 223, 222, 221, 220, 219, 218, 217, 216, 215, 214, 213, 212, 211, 210, 209, 208, 207, 206, 205, 204, 203, 202, 201, 200, 199, 198, 197, 196, 195, 194, 193, 192, 191, 190, 189, 188, 187, 186, 185, 184, 183, 182, 181, 180, 179, 178, 177, 176, 175, 174, 173, 172, 171, 170, 169, 168, 167, 166, 165, 164, 163, 162, 161, 160, 159, 158, 157, 156, 155, 154, 153, 152, 151, 150, 149, 148, 147, 146, 145, 144, 143, 142, 141, 140, 139, 138, 137, 136, 135, 134, 133, 132, 131, 130, 129, 128, 127, 126, 125, 124, 123, 122, 121, 120, 119, 118, 117, 116, 115, 114, 113, 112, 111, 110, 109, 108, 107, 106, 105, 104, 103, 102, 101, 100, 99, 98, 97, 96, 95, 94, 93, 92, 91, 90, 89, 88, 87, 86, 85, 84, 83, 82, 81, 80, 79, 78, 77, 76, 75, 74, 73, 72, 71, 70, 69, 68, 67, 66, 65, 64, 63, 62, 61, 60, 59, 58, 57, 56, 55, 54, 53, 52, 51, 50, 49, 48, 47, 46, 45, 44, 43, 42, 41, 40, 39, 38, 37, 36, 35, 34, 33, 32, 31, 30, 29, 28, 27, 26, 25, 24, 23, 22, 21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0
TRIAL: {'metric_name': 'Ratio', 'metric_value': 1.0, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'Ratio', 'metric_value': 1.0, 'instances_tested': 1, 'conjecture_holds': False}

```

8. Critic adversarial review

Critic verdict: CHALLENGE

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: All tested instances show ratio=1.0, contradicting the conjecture's claim of $\Omega(2^{\{n^2\}})$ asymmetry. | next: Verify if perm_coeff and det_coeff are actually equal in these cases, or if there's a miscalculation in coefficient computation

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	44266
2	propose	ollama_remote	qwen3:8b	0	0	68750
3	preregistration	ollama_remote	qwen3:8b	0	0	24185
4	novelty	ollama_remote	qwen3:8b	0	0	20812
5	novelty	ollama_remote	qwen3:8b	0	0	17669
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10625
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8296
8	critic	ollama_remote	qwen3:8b	0	0	30395
9	judge	ollama_remote	qwen3:8b	0	0	19247

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 244246 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/95c0d42afb3f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/95c0d42afb3f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/95c0d42afb3f.tar.gz` (if generated)